Wrap — ERC-20 to Confidential Token

> ***Nox Protocol — Developer Guide*** *Arbitrum Sepolia Testnet (chainId: 421614)*

Overview

The **Wrap** operation converts a public ERC-20 token (e.g. RLC) into its confidential equivalent (e.g. cRLC) at a 1:1 ratio.

The user sends public tokens to the confidential token smart contract, which **locks** them and mints an equivalent amount of confidential tokens (cTokens) following the ERC-7984 standard.

Use case: A user holds 100 RLC (visible on-chain) and wants to convert them into 100 cRLC (balance encrypted, only visible to the owner).

Architecture

Component	Path / Address
Hook	`hooks/use-wrap.ts`
Modal	`components/modals/wrap-modal.tsx`
ERC-20 ABI	Standard `approve(address, uint256)`
cToken ABI	`lib/confidential-token-abi.ts`
Gas Utility	`lib/gas.ts`
RLC Contract	`0x9923eD3cbd90CD78b910c475f9A731A6e0b8C963`
cRLC Contract	`0x271f46e78f2fe59817854dabde47729ac4935765`

Step-by-Step Flow

```
User                    ERC-20 (RLC)              cToken (cRLC)
  |                         |                          |
  |-- 1. approve(cRLC, N) ->|                          |
  |<--- tx receipt ---------|                          |
  |                         |                          |
  |     [2s cooldown — NoxCompute processing]          |
  |                         |                          |
  |-- 2. wrap(user, N) ----------------------------->  |
  |<--- tx receipt + handle -------------------   |
  |                         |                          |
  | Balance: RLC -= N       |    cRLC += N (encrypted) |
```

Step 1 — Approve (ERC-20)

The user approves the confidential token contract to spend **exactly** the wrap amount. No infinite approval — this is a security choice of the Nox protocol.

```
// Called on the ERC-20 contract (e.g. RLC)
function approve(address spender, uint256 amount) external returns (bool);
```

Parameters:

Param	Value	Description
`spender`	cToken address (`0x271f...765`)	The confidential token contract
`amount`	Exact wrap amount (in wei)	e.g. `parseUnits("100", 9)` for 100 RLC

Implementation:

```
const overrides = await estimateGasOverrides(publicClient);

const approveHash = await writeContractAsync({
  address: token.address,        // RLC address
  abi: erc20Abi,
  functionName: "approve",
  args: [confidentialAddress, parsedAmount],
  ...overrides,
});

await waitForTransactionReceipt(config, { hash: approveHash });
```

After receipt: 2-second cooldown (`await new Promise(r => setTimeout(r, 2000))`) to allow NoxCompute (TEE) to process.

Step 2 — Wrap (cToken)

Calls `wrap()` on the confidential token contract. The amount is sent in **cleartext** — no client-side `encryptInput()` needed for wrap.

```
// Called on the cToken contract (e.g. cRLC)
function wrap(address to, uint256 amount) external returns (bytes32);
```

Parameters:

Param	Value	Description
`to`	User's wallet address	Recipient of the cTokens
`amount`	Exact amount (in wei)	Same as approved amount

Returns: `bytes32` — the new balance handle (encrypted reference to the balance).

What happens inside the contract:

The amount is sent in cleartext in the transaction calldata, but the contract does **not** store it as-is. The **NoxCompute gateway (TEE)** intercepts the operation and:

1. Locks the N ERC-20 tokens in the contract
2. **Encrypts the amount** inside the Trusted Execution Environment
3. Stores the result as an opaque **handle** (`bytes32`) on-chain
4. Returns this handle — which represents the encrypted confidential balance

After wrap, the on-chain balance of cRLC is a handle (e.g. `0x8a3f...b2c1`), not a readable number. Only the owner (or an authorized viewer) can decrypt it using `HandleClient.decrypt(handle)` .

Why is cleartext acceptable here? Because the source ERC-20 balance is already public. Anyone can see that the user had 100 RLC before and 0 after — encrypting the input would add no privacy. The confidentiality starts **after** the wrap, when the balance exists only as an encrypted handle.

> *By contrast, `unwrap()` and `confidentialTransfer()` require client-side encryption via `encryptInput()` because they manipulate confidential balances whose values must never be exposed.*

Implementation:

```
const freshOverrides = await estimateGasOverrides(publicClient);

const wrapHash = await writeContractAsync({
  address: confidentialAddress,  // cRLC address
  abi: confidentialTokenAbi,
  functionName: "wrap",
  args: [address, parsedAmount],
  ...freshOverrides,
});

await waitForTransactionReceipt(config, { hash: wrapHash });
```

State Machine

```
idle → approving → wrapping → confirmed
                        ↘
                       error
```

State	UI	Description
`idle`	Form visible	User selects token + amount
`approving`	Step 1 highlighted, spinner	Approve tx pending
`wrapping`	Step 2 highlighted, spinner	Wrap tx pending
`confirmed`	Success badge + Arbiscan link	Both txs confirmed
`error`	Error message + retry button	Any step failed

Gas Estimation

All transactions use EIP-1559 gas with a **20% buffer** to prevent MetaMask under-estimation on Arbitrum:

```
// lib/gas.ts
export async function estimateGasOverrides(publicClient) {
  const { maxFeePerGas, maxPriorityFeePerGas } =
    await publicClient.estimateFeesPerGas();
  return {
    maxFeePerGas: (maxFeePerGas * 120n) / 100n,
    maxPriorityFeePerGas: (maxPriorityFeePerGas * 120n) / 100n,
  };
}
```

Estimated gas limit: ~150,000 gas units for a wrap operation.

Important Notes

1. **Exact approval only** — Never use infinite approval (`type(uint256).max`). The protocol enforces exact amounts for security.
2. **Cleartext amount, encrypted storage** — The amount is sent in cleartext (no `encryptInput()` needed) because the ERC-20 balance is already public. However, the NoxCompute gateway (TEE) **encrypts it** before storing the confidential balance on-chain as an opaque handle. The confidentiality starts after the wrap.
3. **Handle is ephemeral** — The balance handle returned by `wrap()` changes after every transaction. Always re-read `confidentialBalanceOf(address)` to get the current handle.
4. **2-second cooldown** — Required between approve and wrap because NoxCompute (running in a TEE) needs processing time.
5. **Gas re-estimation** — Gas is estimated fresh before each transaction (approve and wrap separately) because gas prices can change between steps.
6. **Balance invalidation** — After confirmation, React Query caches are invalidated to refresh both public and confidential balances.

Error Handling

Errors are formatted via `formatTransactionError()` in `lib/utils.ts` :

Error	User Message
User rejected tx	"Transaction rejected by user"
Insufficient balance	"Insufficient balance for this operation"
Gas estimation failed	"Gas estimation failed — try again"
Network error	"Network error — check your connection"

Code Section (Dev Mode)

When Developer Mode is enabled, the modal displays the Solidity function signatures:

```
// Step 1: Approve ERC-20
RLC.approve(cRLC_address, amount);

// Step 2: Wrap into confidential token
cRLC.wrap(recipient, amount);
```